



Figure 1: The TVM stack

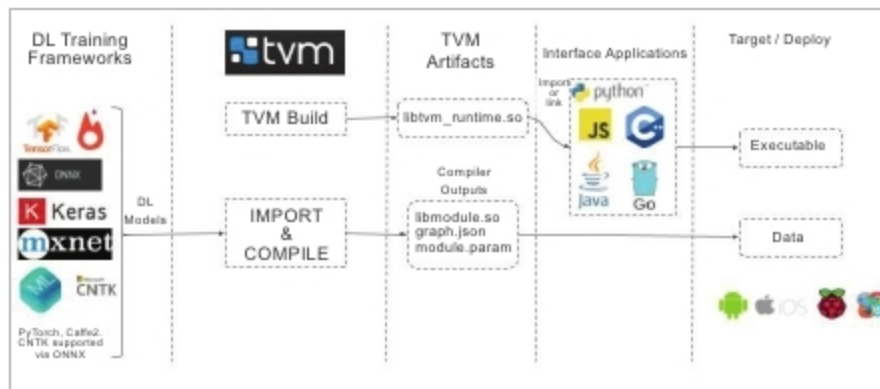


Figure 2: End-to-end flow diagram

The TVM stack comprises multiple layers, as shown in Figure 1. The top and user-facing layer is the framework layer. This is written in Python and contains various import modules for each framework. This layer converts the models from any framework (Tensorflow, Caffe2, etc) to a graph representation of TVM. At the computation graph optimisation layer, the graph representation is optimised by various passes like *precompute prune* which prunes the graph nodes that can be computed at compilation time, the *layout conversion* pass which adds the necessary layout conversion operations (or nodes) across layers if there is a layout mismatch between layers, and a *fusion* pass which joins the computation of multiple nodes into one based on certain rules. These optimisations reduce overall computation costs considerably.

The next layer in the stack is the Tensor compute description which basically generates a computation definition of each node in the graph, based on the inputs. The TOPI sub-module in the stack implements the computes for all the operators. The next layer in the stack is the schedule space and optimisations. This layer is very important in terms of low level as well as hardware-specific optimisations.

Build, integration and deployment

As shown in Figure 2, TVM provides frontends to import trained deep learning models from various frameworks. The TVM compiler outputs a library which contains the operator (layer) compute definitions, the graph represented in JSON format, and a param blob which contains all the parameters of the model. TVM also provides various programming

language binding libraries or packages, which can be imported or linked against, to load the compiled artifacts. Depending on the target hardware configured, the TVM runtime can be as small as approximately 300kB.

To demonstrate the end-to-end capabilities of the TVM compiler stack here, let us look at an example of importing, compiling and running a TensorFlow vision model, MobilenetV1, on x86 and NVIDIA (CUDA) targets.

Step 1: The setup

TVM can be setup by building from source or using Docker. The simple steps given below can set up the environment for us. Please refer to <https://docs.tvm.ai/install/index.html> for other ways of setting up.

Download the source as follows:

```
test@test-pc:~$ git clone --recursive https://github.com/dmlc/tvm
```

Build and run Docker using the following code:

```
test@test-pc:~$ cd tvm
test@test-pc:~$ ./docker/bash.sh tvmai/demo-cpu
```

You may choose to launch Jupyter, if needed, inside Docker, as follows:

```
test@test-pc:~$ jupyter notebook
```

One can verify the setup by just importing TVM as shown below, in Notebook or in a Python shell.

```
import tvm
```

Step 2: Downloading the TensorFlow MobilenetV1 model

Various TensorFlow official models can be found at <https://github.com/tensorflow/models/tree/master/research/slim>. Use the following link to download the MobilenetV1 official model from TensorFlow: http://download.tensorflow.org/models/mobilenet_v1_2018_02_22/mobilenet_v1_1.0_224.tgz.

Extract the downloaded archive and look for *mobilenet_v2_1.0_224_frozen.pb*. This is the Protobuf format for a frozen model after training. Latest version of TVM also supports importing Tensorflow saved bundle. Model-specific information like input shapes, input and output node names is needed to compile a model from TensorFlow. For TensorFlow, this information is available in the file below, from Mobilenet. The input shape for Mobilenet is (1, 244, 244, 3), which is the image resolution

```
cat mobilenet_v1_1.0_224_info.txt
Model: mobilenet_v1_1.0_224
```